

Decisão Estratégica no Pôquer com IA baseada em Expressões Faciais, Simulações de Monte Carlo e análise do jogo

João Pedro de Souza Costa Ferreira ¹, Pedro Nomura Picchioni ¹, Victor Vaglieri de Oliveira ¹, Alcides Teixeira Barboza Junior ¹

¹ Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie – São Paulo, SP – Brasil

{10400720,10401616,10400787}@mackenzista.com.br

alcides.barboza@mackenzie.br

Resumo. *Este trabalho abordou o desenvolvimento de uma Inteligência Artificial (IA) aplicada ao jogo de pôquer. O pôquer foi escolhido devido à sua complexidade e à necessidade de tomar decisões sob incerteza. O objetivo principal foi investigar se a análise de reconhecimento de expressões ajudaria na tomada de decisão da IA, melhorando uma estratégia que é baseada em probabilidade e análise do jogo. A metodologia envolveu a aplicação de modelos probabilísticos do Método de Monte Carlo para calcular as chances de vitória e algoritmos de reconhecimento facial de Transfer Learning para interpretar o estado emocional do oponente. O desenvolvimento da IA foi de forma modular a qual integra os dados matemáticos e emocionais em uma fórmula de decisão. Para avaliar a performance, a IA foi testada em 60 partidas simuladas e comparada a 60 testes em um cenário sem leitura facial. Os resultados demonstraram que o módulo de leitura facial foi a variável que reverteu resultados de prejuízo em lucro, que valida a hipótese central do trabalho.*

Palavras-chave: *Pôquer; Probabilidade; Inteligência Artificial; Expressões Faciais; Monte-Carlo.*

Abstract. *This work addressed the development of an Artificial Intelligence (AI) applied to the game of poker. Poker was chosen due to its complexity and the need to make decisions under uncertainty. The main objective was to investigate whether facial recognition analysis would aid in the AI's decision-making, improving a strategy based on probability and game analysis. The methodology involved applying probabilistic Monte Carlo models to calculate the odds of winning and Transfer Learning facial recognition algorithms to interpret the opponent's emotional state. The AI was developed in a modular way, integrating mathematical and emotional data into a decision formula. To evaluate performance, the AI was tested in 60 simulated games and compared to 60 tests in a scenario without facial recognition. The results demonstrated that the facial recognition module was the variable that reversed losses into profits, validating the central hypothesis of the work.*

Keywords: *Poker; Probability; Artificial Intelligence; Facial Expressions; Monte-Carlo.*

1. Introdução

Este trabalho tem como objetivo desenvolver uma inteligência artificial (IA) para jogar pôquer, um jogo amplamente difundido que envolve tanto cálculos probabilísticos quanto reconhecimento de expressões (FEINLAND et al., 2022). O objetivo central, no entanto, não é apenas criar um *bot* com a IA integrada, mas investigar se a tomada de decisão baseada em probabilidade pode ser significativamente melhorada pela adição da leitura das expressões faciais do oponente.

A escolha do pôquer como foco de desenvolvimento se justifica pela sua complexidade e pela necessidade de tomar decisões sob incerteza, já que os jogadores não têm conhecimento de todas as cartas em jogo. O fator humano torna o jogo ainda mais imprevisível, o que cria o ambiente ideal para testar a hipótese central deste trabalho e aplicar técnicas de IA sofisticadas. Como apontam Brown e Sandholm (2019), o desenvolvimento de algoritmos para pôquer representa um avanço significativo no entendimento de interações estratégicas complexas em cenários incertos.

Jofilsan et al. (2021) apontam que essa abordagem também permite identificar padrões emocionais e revelar tendências comportamentais no decorrer do jogo, como propensão ao blefe e nível de confiança ou insegurança diante de determinadas situações. Dessa forma, o reconhecimento das expressões faciais não só contribui para uma análise instantânea do jogador, mas também uma análise mais detalhada de seu perfil conforme ele joga mais vezes. Esses índices de intensidade emocional foram, portanto, utilizados para que a IA tome uma melhor decisão.

Os autores Feinland et al. (2022) em *Poker Bluff Detection Dataset Based on Facial Analysis* utilizaram vídeos de torneios profissionais de pôquer para coleta de dados autênticos sem a necessidade de simulações artificiais. Os autores compararam modelos de classificação baseados em redes neurais convolucionais (Convolutional Neural Network - CNN), nesse caso o *OpenFace 2.0*, uma biblioteca que usa *Transfer Learning*, com outras técnicas. Os resultados indicam que, especialmente no modelo binário (blefe claro contra não blefe), é possível detectar sinais de blefe com alta acurácia por meio da análise facial.

Embora o escopo deste estudo seja o pôquer, ele pode ser aplicado a situações reais onde a interpretação de expressões e comportamentos humanos é essencial, como em negociações comerciais ou sistemas de segurança, por exemplo.

2. Referencial Teórico

2.1 Reconhecimento facial com *Transfer Learning*

A parte inicial deste trabalho consiste no uso do *Transfer Learning* para o reconhecimento de expressões faciais no contexto do pôquer. O *Transfer Learning* permite reutilizar modelos previamente treinados em grandes bases de dados, tornando viável a aplicação de Aprendizado Profundo mesmo em cenários onde há escassez de

dados rotulados (ALHANAEE et al., 2021; JOFILSAN et al., 2021). Dessa forma, ao invés de treinar técnicas como CNN do zero, que exigiria um grande volume de dados e poder computacional (PEI; SHAN, 2019; JIN; CRUZ; GONÇALVES, 2020), se opta pelo *Transfer Learning*, já que o mesmo melhora a eficiência e a generalização do modelo.

Sob a ótica técnica, o *Transfer Learning* é uma abordagem de Aprendizado de Máquina na qual um modelo treinado para uma tarefa específica é reaproveitado como ponto de partida para uma segunda tarefa (ALHANAEE et al., 2021), o que a torna enquadrada no contexto do projeto, uma vez que os resultados foram integrados para a criação de uma IA. Bibliotecas como *DeepFace*, exemplificam o uso de *Transfer Learning* ao incorporarem modelos previamente treinados para tarefas de reconhecimento facial, permitindo a adaptação desses modelos a novos contextos com menor necessidade de dados específicos (SERENGIL; OZPINAR, 2020).

Alhanaee et al. (2021) comentam que essa técnica se destaca por ser uma solução eficaz para problemas que exigem análise de grandes volumes de dados em tempo real. No contexto do projeto, não haverá necessidade de analisar grandes volumes de dados, porém, o *Transfer Learning* oferece otimização específica, melhora de tempo e melhora na performance (ALHANAEE et al., 2021; JOFILSAN et al., 2021). Estas características tornam o *Transfer Learning* ideal para esse contexto, pois ela é adaptada às necessidades do projeto garantindo que a IA possa agir rapidamente, levando em conta expressões faciais dos oponentes em tempo real.

Para Jin, Cruz e Gonçalves (2020), apesar da alta precisão das CNNs, um dos seus principais desafios é a necessidade de um banco de dados específico para o pré-treinamento. Isso significa que, como apontado por Pei e Shan (2019) e Jin, Cruz e Gonçalves (2020), para melhor uso dessa técnica, são utilizadas bases de dados específicas, tornando-a pouco otimizável e sendo menos compatível com o projeto, que utilizará reconhecimento dinâmico, possibilitando analisar qualquer rosto.

Além disso, as CNNs exigem dados coletados previamente. Isso se torna um grande obstáculo, pois modelos como esses necessitam de pré-treinamento de dados o que demanda não apenas grandes *datasets*, mas também infraestrutura computacional avançada que não está no escopo do trabalho (JIN; CRUZ; GONÇALVES, 2020).

Diante desses desafios, a biblioteca *DeepFace*, proposta por Serengil e Ozpinar (2020), emerge como a solução ideal para o projeto, pois utiliza *Transfer Learning* com modelos pré-treinados (como *VGG-Face*, *Facenet* e *OpenFace*), o que permite a análise de expressões faciais sem a necessidade de treinamento a partir do zero. Isso resolve o problema da dependência de grandes *datasets* específicas, já que a biblioteca aproveita conhecimento adquirido em bases genéricas de reconhecimento facial (ex.: *Labeled Faces in the Wild* - LFW), consumindo baixo poder computacional, ideal para o projeto.

2.2 Algoritmo de Monte Carlo

Para lidar com probabilidades em cenários complexos, utiliza-se o algoritmo de Monte Carlo, que estima a chance de vitória com base nas cartas disponíveis (LIMA; MADEIRA, 2017; SANTOS; BERNARDINO, 2017). Como descrito por Brown e Sandholm (2019), essa estimativa permite que a IA tome decisões com base em simulações estatísticas, considerando o cenário parcial do jogo. O algoritmo realiza várias simulações possíveis com as cartas conhecidas, calcula quantas resultam em vitória e, a partir disso, gera a probabilidade de ganho. Esse método é aplicado em jogos com incerteza, como o pôquer, onde as cartas do oponente e da mesa não são visíveis.

Entre os tipos de algoritmos baseados em Monte Carlo, destacam-se: o Monte Carlo Clássico, o Monte Carlo com Heurísticas e o *Monte Carlo Tree Search* (MCTS). O Monte Carlo Clássico realiza simulações completas do jogo usando apenas as informações conhecidas, como as cartas do jogador e as cartas da mesa abertas. As cartas ocultas, como as do oponente e as cartas fechadas da mesa, são sorteadas aleatoriamente em cada simulação a partir do conjunto das cartas restantes do baralho. Ao repetir esse processo, o algoritmo calcula a frequência de vitórias e, assim, estima a probabilidade de ganhar a mão em dado momento. Esse método não utiliza estruturas como árvores ou funções de avaliação e, por isso, tem implementação um pouco simples, menor custo computacional e é mais eficaz em jogos com informações parciais (LIMA; MADEIRA, 2017).

O Monte Carlo com Heurísticas, por outro lado, modifica o processo de simulação ao incorporar funções de avaliação pré-definidas que guiam a escolha de ações durante as simulações. Em vez de realizar amostragens puramente aleatórias, ele prioriza jogadas consideradas mais promissoras com base em conhecimento prévio do jogo (GELLY; WANG; MUNOS; TEYTAUD, 2006). Embora isso indique que pode acelerar a convergência para boas estimativas, esse tipo de abordagem depende da definição e qualidade das heurísticas. Se essas funções forem baseadas em premissas incorretas, o algoritmo pode produzir resultados enviesados (CHASLOT et al., 2008).

O *Monte Carlo Tree Search* (MCTS) é mais sofisticado: ele utiliza uma árvore de decisões que é expandida dinamicamente com base nas simulações realizadas. O algoritmo equilibra a exploração de novos caminhos e a exploração de caminhos promissores já conhecidos. Essa abordagem é eficaz em jogos determinísticos com grande número de possibilidades, como o xadrez ou o Go, nos quais o histórico de decisões pode indicar com precisão estratégias futuras. No entanto, no pôquer, que possui informações incompletas e de aleatoriedade, o MCTS enfrenta limitações. Sua estrutura depende de dados parciais e requer maior capacidade computacional para manter e expandir a árvore de decisões. Comparado ao Monte Carlo Clássico, o MCTS exige mais recursos e pode não oferecer ganhos proporcionais em precisão ou desempenho, especialmente em ambientes de incerteza (COULOM, 2006; KOCSIS; SZEPESVÁRI, 2006).

Para este projeto, foi utilizado o Monte Carlo Clássico. A decisão se baseia em sua implementação direta (menor exigência computacional e independência de estruturas complexas) e adequação ao pôquer, que requer decisões simples como apostar ou não apostar, cobrir uma aposta ou sair da rodada, diferentemente do xadrez, que contém diversas possibilidades de ação. Isso permite que ele funcione de forma integrada com o reconhecimento facial, servindo como uma das entradas para a tomada de decisão da IA durante o jogo. Em um ambiente com decisões que precisam ser rápidas e informações parciais, o Monte Carlo Clássico apresenta melhor relação entre simplicidade e desempenho (SARAIVA JÚNIOR; TABOSA; COSTA, 2011).

2.3 Artigos relacionados

O estudo desenvolvido por Alhanaee et al. (2021) mostra a eficiência de técnicas avançadas de reconhecimento de expressões faciais, incluindo *Transfer Learning* que permite a identificação e categorização de padrões emocionais. A capacidade de detectar expressões e relacioná-las com jogadores pode aprimorar a tomada de decisão da IA, fornecendo uma base para avaliar o comportamento adversário no jogo.

Nesse contexto, Jin, Cruz e Gonçalves (2020) e Alhanaee et al. (2021) demonstraram que, ao aplicar *Transfer Learning* em redes neurais convolucionais pré-treinadas (como *SqueezeNet*, *GoogleNet* e *AlexNet*), é possível alcançar altas taxas de precisão na validação, ajustando os modelos a conjuntos de dados específicos.

A literatura recente fornece bases importantes para a escolha metodológica adotada neste trabalho. Brown e Sandholm (2019) demonstram como técnicas probabilísticas e análise estatística podem atingir desempenho sobre-humano no pôquer, mesmo em cenários de informação incompleta. Embora seu sistema utilize abordagens complexas baseadas em equilíbrio, o estudo reforça a viabilidade de soluções que tratam a incerteza de forma sistemática. Por outro lado, Coulom (2006) e Kocsis e Szepesvári (2006) introduzem avanços em algoritmos baseados em Monte Carlo, discutindo mecanismos para tomada de decisão por simulação e aprendizado adaptativo. Esses trabalhos fundamentam a ideia de que métodos baseados em amostragem são eficazes em jogos de decisão sequencial.

Complementarmente, o estudo de Feinland et al. (2022) fornece evidências de que expressões faciais estão relacionadas com o comportamento estratégico dos jogadores, indicando que sinais emocionais podem ser utilizados como uma fonte adicional de informação. Em conjunto, esses estudos sustentam a proposta de integrar simulações probabilísticas com leitura facial para melhorar a performance da IA no pôquer.

3. Metodologia

O processo metodológico deste trabalho foi estruturado para abranger desde o desenvolvimento dos módulos do *bot* com base na IA até a coleta e análise de dados.

A Inteligência Artificial proposta integrou as duas técnicas principais: o reconhecimento de expressões com *DeepFace* (baseada no *Transfer Learning*) e a análise probabilística baseada no método de Monte Carlo, o que permitiu o fornecimento das duas fontes de dados principais para a tomada de decisão da IA.

Essas duas fontes de informação foram processadas e integradas separadamente, permitindo que a IA possa combinar sinais das expressões dos adversários com estimativas probabilísticas para jogar de forma mais estratégica. Junto a isso, a IA leva em conta variáveis contextuais da partida, como níveis dos *blinds*, fichas e mãos jogadas, que são descritas na seção 3.3.3.

Esta seção detalha as etapas de desenvolvimento, as ferramentas utilizadas, a arquitetura do sistema, o detalhamento do motor de decisão e os desafios enfrentados.

3.1. Processo de desenvolvimento e análise

O desenvolvimento da IA foi dividido em quatro etapas principais, cada uma responsável por uma camada do comportamento do *bot*:

1. Inicialmente, foi implementado o reconhecimento facial com a biblioteca *DeepFace*, utilizando a função *DeepFace.analyze()* para identificar emoções durante a partida. A leitura é feita quadro a quadro, mas foi otimizada para capturar em intervalos de 5 segundos, reduzindo travamentos e obtendo uma média mais estável das emoções.
2. Em seguida, foi desenvolvido o cálculo de probabilidade do método de Monte Carlo. Nela, o código embaralha as cartas restantes do baralho, simula diversos resultados com base nas cartas conhecidas e calcula a taxa de vitória do *bot* com base nas simulações. Essa informação é usada para alimentar a variável de decisão do *bot*.
3. A etapa seguinte focou na avaliação e ajuste na interpretação das emoções. Se observou que anteriormente ao aceitar a emoção com a maior pontuação gerava inconsistências. Para a correção, foi aplicado um limiar de confiabilidade onde se passou a exigir um valor mínimo de 60% para considerar uma emoção válida. Esse ajuste tornou a leitura mais confiável e menos sensível a ruídos visuais, descartando leituras incertas que poderiam prejudicar a IA.
4. Aplicação da fórmula de decisão, inicialmente planejada, previa uma análise complexa de fatores como o de agressividade, em momentos ofensivos com variáveis de diferença de fichas e situações defensivas com variáveis que consideravam a diferença de apostas em *blinds*.

Devido ao tempo de desenvolvimento e à complexidade da calibração, a fórmula foi modificada para uma versão mais direta, em que as variáveis como a de Agressividade e diferença de *blinds* foram removidas para manter o foco principal do trabalho, pois exigiriam tanto um sistema complexo de rastreamento do comportamento do oponente ao longo do jogo quanto um aumento na complexidade da fórmula. A modificação resultou em uma equação final que integra a Probabilidade de Monte Carlo, a Leitura Facial e o Fator de Apostas, cujo detalhamento matemático completo encontra-se na Seção 3.3.3. Esses parâmetros foram ajustados conforme os resultados das simulações e as reações emocionais do jogador, determinando as ações do *bot* (*Call*, *Fold*, *Bet*, *Check*) de acordo com faixas de decisão e a situação atual da jogada.

3.2. Ambiente, ferramentas, arquiteturas do sistema

O sistema foi desenvolvido em Python, utilizando as bibliotecas *Pygame* (interface e controle de jogo), *DeepFace* (reconhecimento facial e análise emocional), *PokerLib* (manipulação de cartas e simulação de mãos), *random* e *math* (operações probabilísticas e numéricas de montecarlo), *openCV* (captura e tratamento de imagens) e *numpy* (processamento de vetores e operações matriciais).

O software foi estruturado como um sistema modular, onde a IA funciona como um módulo externo ao ambiente principal do jogo, por meio da biblioteca *Pygame*. O ambiente em *Pygame* é responsável por toda a lógica do jogo de pôquer *Texas Hold'em*, incluindo interface gráfica, controle de turnos, distribuição de cartas e apostas. O desenvolvimento dessa lógica seguiu uma estrutura de um *loop* principal baseado em máquina de estados. Esse *loop* gerencia os eventos de entrada e atualiza os elementos visuais na tela conforme a etapa atual da rodada. A interface gráfica resultante é apresentada na Figura 1.



Figura 1 – Interface gráfica do jogo de pôquer desenvolvido em Pygame

Fonte: Autores

Por simplicidade e controle, o jogo é sempre no formato jogador contra *bot*, o que permite focar na integração entre a IA e o ambiente do jogo sem variáveis externas introduzidas por múltiplos jogadores.

A IA, representada na Figura 2, foi implementada separadamente. Em seu turno, ela recebe os dados da mesa (cartas e fichas). Essas informações são processadas em conjunto com os resultados da análise facial (verde), da simulação de Monte Carlo (rosa) e do estado do jogo (vermelho), sendo então integradas por uma fórmula de decisão (preto), que define a jogada a ser executada na rodada dentro do ambiente do jogo.

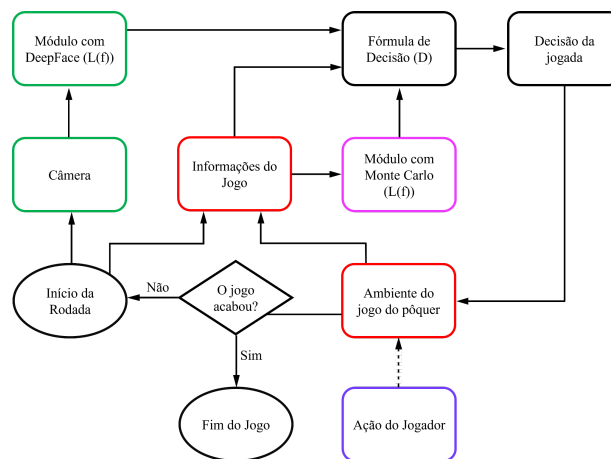


Figura 2 – Diagrama do sistema da IA

Fonte: Autores

3.3. Integração e Calibração do Motor de Decisão

Esta subseção detalha a implementação dos principais módulos que compõem o motor de decisão do *bot*: a simulação Monte Carlo e a interpretação das expressões faciais.

3.3.1. Calibração do Método de Monte Carlo

O Método de Monte Carlo foi implementado para estimar as probabilidades de vitória de cada mão no jogo de pôquer. A simulação consiste em gerar aleatoriamente várias combinações possíveis de cartas usando funções da biblioteca *PokerLib* e do módulo *random*. Cada cenário é avaliado para determinar o desfecho da mão, derrota ou não derrota (para este trabalho, empates foram considerados como positivos para a continuação do *bot* na partida), e a probabilidade de vitória é calculada com base na frequência relativa desses resultados. A calibração do Monte Carlo foi feita ajustando o número de simulações por mão até que os resultados se estabilizassem, garantindo um equilíbrio entre precisão e desempenho computacional. Valores acima de 10.000 não

demonstraram mudanças significativas na probabilidade calculada e apresentavam problemas de performance.

3.3.2. Processamento de Expressão Facial

Segundo Serengil e Ozpinar (2020), *DeepFace* é um *framework* híbrido de reconhecimento facial que integra modelos de última geração. Cada modelo é uma rede neural convolucional (CNN) treinada em grandes *datasets*.

O *pipeline* de reconhecimento facial do *DeepFace* segue cinco etapas automatizadas:

1. Detecção;
2. Alinhamento (padroniza a pose);
3. Normalização (ajusta iluminação);
4. Representação (extraí as características faciais e as converte em vetores numéricos únicos via CNNs);
5. Verificação (compara *embeddings*).

A função `stream` da biblioteca gerencia o acesso à webcam e detecção de rostos, permitindo a análise contínua de atributos. O algoritmo avalia a posição dos olhos, sobrancelhas, boca e músculos faciais, retornando índices que variam entre 0,00 a 100,00 para cada emoção: felicidade, tristeza, raiva, surpresa, medo, nojo e neutro (SERENGIL; OZPINAR, 2020). A biblioteca resolve, assim, desafios de dependência de dados (usando pré-treinamento) e custo computacional.

As expressões faciais do jogador, capturadas pela webcam com o auxílio da biblioteca *DeepFace*, são tratadas por meio desses passos:

- Coleta de *frames* durante um intervalo de tempo (duração).
- Extração das probabilidades de cada emoção em cada *frame*.
- Desconsideração da emoção “neutra” para focar em sinais emocionais relevantes.
- Cálculo da média das probabilidades de cada emoção durante o tempo.
- Identificação da emoção dominante, isto é, aquela com maior probabilidade média.

Após identificar a emoção dominante, o sistema converte essa probabilidade em um valor numérico, representando o seu impacto no jogo. A aplicação desse valor na fórmula de decisão depende da intensidade da detecção: caso a probabilidade da emoção seja maior que o limiar pré-definido, o valor é aplicado integralmente. Caso contrário, o impacto é calculado proporcionalmente à confiança da detecção. Esse processo associa cada emoção a um peso fixo, baseado em interpretações comportamentais, como segue:

Emoção	Felicidade	Tristeza	Raiva	Medo	Aversão	Surpresa
Impacto Numérico	-30	+45	+45	+35	+40	-50

Tabela 2 – Impacto numérico de acordo com a expressão

Fonte: Autores.

Por exemplo: caso reconheça alegria (*happy*) com 10% de intensidade, o impacto final seria $-30 \times 0,10 = -3$. Se estivesse acima do limiar (60%) se aplica o valor integral de -30.

Os valores de "Impacto Numérico" apresentados na Tabela 2 são heurísticas de calibração. Os valores exatos foram determinados por meio de testes e observações durante o desenvolvimento. No entanto, a lógica de direção (positiva ou negativa) e a magnitude (alta ou baixa) por trás dos números foi embasada no Modelo Circumplexo de Afeto de James Russell (1980). Este modelo mapeia emoções em um plano cartesiano sendo o eixo horizontal o de Valência (o eixo prazeroso-desprazeroso) e o eixo vertical em Ativação (o eixo calmo-estimulante), conforme ilustrado na Figura 3.

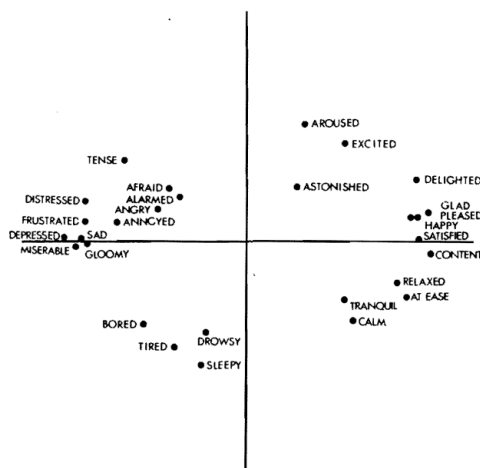


Figura 3 – O Modelo Circumplexo de Afeto

Fonte: Russell (1980)

A heurística foi desenvolvida como uma função de Valência, que traduz o estado emocional do oponente em um modificador de decisão para o *bot*. A lógica teórica de calibração, baseada em Russell (1980), foi a seguinte:

1. Emoções de Baixa Valência (Negativas): Emoções como *Sad*, *Angry*, *Fear* e *Disgust* localizam-se no lado de baixa valência (desprazer) do modelo de Russell. Neste trabalho, assumiu-se que tais expressões indicam insatisfação com as cartas. Portanto, foram atribuídos valores positivos, que aumentam a

confiança do *bot*. O valor empírico de *Disgust* (+40) foi definido como o mais alto, por ser interpretada como a reação de aversão mais forte e por consequência a informação de fraqueza mais confiável.

2. Emoções de Alta Valência (Positivas): A emoção *Happy* localiza-se no lado de alta valência. Considerou-se que esta emoção sinaliza a posse de uma mão forte. Consequentemente, foi atribuído um valor negativo (-30), que subtrai da “confiança” do *bot*.
3. Emoções de Valência Ambígua (Surpresa): A emoção *Surprise* possui alta ativação, mas sua valência pode ser positiva ou negativa. Nesta heurística, optou-se por classificá-la como um indicador de jogo vencedor. Por isso, foi tratada como uma emoção de alta valência e recebeu um valor negativo (-50).

3.3.3. Detalhamento da Fórmula de Decisão

A fórmula geral de decisão (D) integra a probabilidade de vitória (Pmc), a leitura facial ($L(f)$) e os fatores situacionais ($F(x)$). Essa estrutura foi formulada para este projeto, desenvolvida para consolidar os resultados das técnicas de probabilidade e reconhecimento facial, somados às variáveis situacionais do jogo.

3.3.3.1. Fórmula geral

$$D = Pmc + L(f) + F(x)$$

Onde:

- D : Decisão final - Pontuação total usada para determinar a ação do *bot*.
- Pmc : Probabilidade Monte Carlo - Valor (0-100) da simulação de vitória.
- $L(f)$: Leitura facial - Índice obtido pelo reconhecimento emocional (Tabela 2).
- $F(x)$: Fatores situacionais - Contexto defensivo ou ofensivo da rodada.

3.3.3.2. Fatores situacionais

$$F(x) = Cf + S * Rf + (S - 1) * Cp$$

Onde:

- Cf : Comparação de fichas - Proporção de fichas *bot*/oponente variando entre V positivo e negativo, a qual o valor de V será explicado na seção 3.4.
- S : Situação de aposta
 - $S = 1 \rightarrow$ oponente apostou (situação defensiva)
 - $S = 0 \rightarrow$ oponente não apostou (situação ofensiva)

- *Rf*: Risco financeiro - Custo para pagar a aposta versus o pote total. (Usado apenas se $S=1$).
- *Cp*: *Check* do oponente - Indica se o oponente deu *check* na vez dele. (Usado apenas se $S=0$).

3.3.3.3. Lógica de Ação Baseada da variável D

Os limites de decisão, que determinam a ação final do *bot*, foram simplificados para garantir uma resposta clara e direta. A origem e justificativa desses limiares são explicadas na Seção 3.4.

- Se houve aposta do oponente ($S = 1$):
 - $D \geq 50$: *Call()* — (Pagar aposta)
 - $D < 50$: *Fold()* — (Desistir)
- Se não houve aposta ($S = 0$):
 - $D \geq 50$: *Bet()* — (Apostar)
 - $D < 50$: *Check()* — (Passar a vez)

3.4. Determinação dos valores para análise do jogo

Os valores numéricos utilizados nos Fatores Situacionais (variável V) e nos limites de decisão (variável D) foram estabelecidos por meio de duas metodologias distintas: análise de ponto de equilíbrio e calibração empírica.

3.4.1. Justificativa do Limiar de Decisão

A lógica é que a variável *Pmc* (Probabilidade Monte Carlo) fornece a probabilidade de vitória de 0 a 100. Um *Pmc* de 50 representa, portanto, uma chance estatisticamente neutra (50/50) de vitória. Este valor foi adotado como a base para a variável D total.

- $D < 50$: Indica uma expectativa líquida negativa (somando probabilidade, emoção e situação), justificando uma ação passiva (*Fold/Check*).
- $D \geq 50$: Indica uma expectativa líquida neutra ou positiva, justificando uma ação ativa (*Call/Bet*).

3.4.2. Determinação do Peso Situacional da variável V

O valor da variável V citado anteriormente que define o peso dos fatores da variável *Cf* (Comparação de Fichas), variável *Rf* (Risco Financeiro) e variável *Cp* (*Check* do *Player*), foi determinado por meio de calibração empírica. Foi desenvolvido

um *script* de simulação separado, executado independentemente do jogo principal, que colocou o *bot* para jogar contra uma versão de si mesmo durante 100 mãos, 5 vezes para cada dupla de valores, mudando apenas o V de cada versão.

Neste ambiente de testes, diferentes valores para V (3, 5, 8, 10, 12) foram iterativamente testados. O valor 10 demonstrou a maior taxa de vitória agregada durante essas simulações e foi o selecionado para a implementação final. Esta abordagem garantiu que esses fatores tivessem o peso mais otimizado possível para o andamento do trabalho atual.

3.5. Desafios Enfrentados no Reconhecimento Facial

Durante o desenvolvimento do módulo de reconhecimento emocional, foram identificados alguns desafios técnicos e metodológicos, o que levou a ajustes na sua implementação.

3.5.1. Baixa eficiência no ciclo de captura da webcam

Inicialmente, o sistema realizava a sequência completa de “abrir a câmera → reconhecer emoção → fechar a câmera” a cada jogada. Esse processo se mostrou lento, o que prejudicou as partidas. Como solução, a câmera passou a ser aberta apenas uma vez no início do jogo, para assim ser possível realizar múltiplas leituras ao longo do jogo e sendo encerrada ao final.

3.5.2. Tempo insuficiente para análise emocional

O tempo de captura das emoções era curto, que resultava em leituras instáveis e ruins. Para a melhora, o período de análise foi ampliado para 5 segundos, o qual permite coletar várias amostras de expressão facial e calcular uma média das probabilidades obtidas para uma melhor resposta.

3.5.3. Mudança no critério de validação da emoção dominante

No início do projeto, a emoção com maior probabilidade era automaticamente considerada a emoção dominante. Esta estratégia se mostrou imprecisa em situações com valores muito próximos entre as emoções. Logo foi trocada a lógica para utilizar um limiar mínimo de confiança onde somente emoções que ultrapassam esse valor são consideradas válidas. Caso nenhuma emoção atinja o limiar então se aplica uma ponderação proporcional da maior probabilidade.

3.5.4. Testes com bibliotecas alternativas

Durante o desenvolvimento, a biblioteca FER (*Facial Emotion Recognition*) foi testada como alternativa à *DeepFace*, já que a mesma apresentava leituras imprecisas, caracterizadas por oscilações repentinas nos valores de confiança das expressões entre os frames. No entanto, os seus resultados foram similares em alguns momentos e

inferiores em outros em relação à precisão e estabilidade, levando à continuação do uso do *DeepFace* como solução principal.

3.5.5. Condições necessárias para funcionamento adequado

O reconhecimento facial exige algumas condições para obter resultados confiáveis e, caso não sejam cumpridas, terá como resultado uma piora nas suas jogadas. Dentro delas são:

- Iluminação adequada no rosto do jogador;
- Ausência de obstruções como óculos escuros, mãos ou cabelos cobrindo os olhos;
- Versão do Python 3.10 mostrou melhor compatibilidade;
- Necessidade de instalar as bibliotecas *DeepFace* com *tf-keras*;
- No ambiente Windows, o sistema utiliza automaticamente a primeira webcam detectada.

3.6. Procedimento de Teste e Coleta de Dados

Foram realizadas 120 partidas simuladas para avaliar o comportamento do motor de decisão. A avaliação foi feita pelos autores (usuários de teste), sem categorização de nível de jogador, a qual cada autor jogou 20 partidas com o módulo de reconhecimento facial ligado e 20 partidas com o mesmo desligado. Os dados foram registrados por meio do *log* do terminal, transcritos manualmente para planilhas Excel e interpretados para analisar tendências e o impacto da integração.

4. Resultados e Discussão

Esta seção apresenta e analisa os dados coletados nas 120 partidas simuladas, sendo 40 para cada autor, conforme a metodologia descrita na Seção 3.6. O objetivo foi avaliar o comportamento do motor de decisão e quantificar o impacto da integração do módulo de reconhecimento facial nos resultados.

4.1. Análise Comparativa de Desempenho

A primeira etapa da análise de resultados foca em quantificar o impacto direto do módulo de reconhecimento facial. Para isso, são comparados os resultados de 60 partidas com o módulo $L(f)$ ativo contra os resultados de 60 partidas simuladas com o módulo desligado.

Autores	Número de partidas	Vitórias do Bot	Derrotas do Bot	Fichas finais do Bot	Fichas finais do Autor
Jogador 1	20	10	10	5500	4500

Autores	Número de partidas	Vitórias do <i>Bot</i>	Derrotas do <i>Bot</i>	Fichas finais do <i>Bot</i>	Fichas finais do Autor
Jogador 2	20	9	11	5450	4550
Jogador 3	20	11	9	5950	4050

Tabela 3 – Resumo Consolidado das Partidas Simuladas com Módulo L(f) ligado

Fonte: Autores

Autores	Número de partidas	Vitórias do <i>Bot</i>	Derrotas do <i>Bot</i>	Fichas finais do <i>Bot</i>	Fichas finais do Autor
Jogador 1	20	8	12	3450	6550
Jogador 2	20	11	9	4750	5250
Jogador 3	20	8	12	5500	4500

Tabela 4 – Resumo Consolidado das Partidas Simuladas com Módulo L(f) desligado

Fonte: Autores

As Tabelas 3 e 4 apresentam os dados onde é possível fazer comparação direta entre o desempenho do *bot* com o módulo de reconhecimento facial que será mencionado como cenário A (Tabela 3) e sem ele como cenário B (Tabela 4).

Na análise do cenário A, o *bot* atingiu uma taxa de vitória por partida de 50%. Em relação à lucratividade, o *bot* terminou com saldo positivo contra os três autores: Jogador 1 (+500 fichas), Jogador 2 (+450 fichas) e Jogador 3 (+950 fichas).

A análise do Cenário B apresenta um resultado oposto. O *bot* obteve uma taxa de vitória de 45%. O *bot* teve prejuízo contra dois dos três autores: Jogador 1 (-1.550 fichas) e Jogador 2 (-250 fichas), e um ganho contra Jogador 3 (+500 fichas).

A comparação dos resultados de ambos os cenários mostra o impacto do módulo *L(f)* onde o cenário A resultou em um lucro de +1.900 fichas, enquanto o cenário B resultou em um prejuízo de -1.300 fichas.

4.2. Análise Gráfica da Evolução do Saldo de Fichas

Enquanto a Seção 4.1 analisou os saldos finais, os gráficos 1, 2 e 3 a seguir detalham a evolução do desempenho partida a partida para cada um dos três autores. Em

cada gráfico, a linha azul representa o cenário A e a linha laranja representa o cenário B. O eixo vertical mostra o saldo de fichas, e o eixo horizontal mostra as 20 partidas jogadas. O ponto zero marca o início do jogo, onde ambos começam com 5.000 fichas.

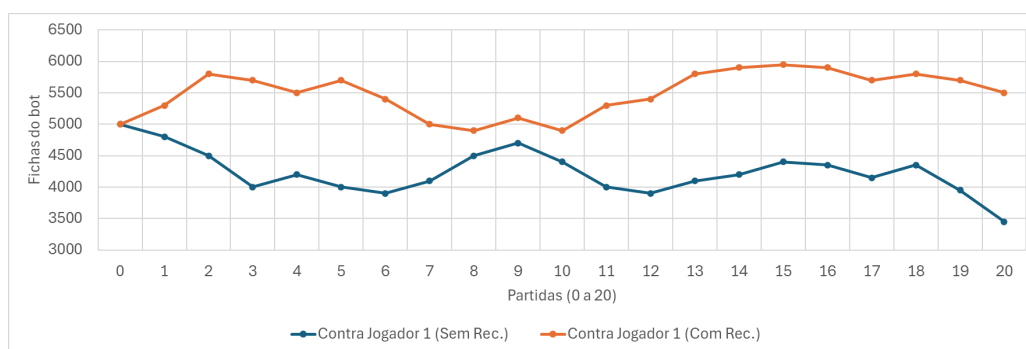


Gráfico 3 – Comparativo de Desempenho contra Jogador 1 (Com Reconhecimento Facial vs. Sem Reconhecimento Facial)

Fonte: Autores

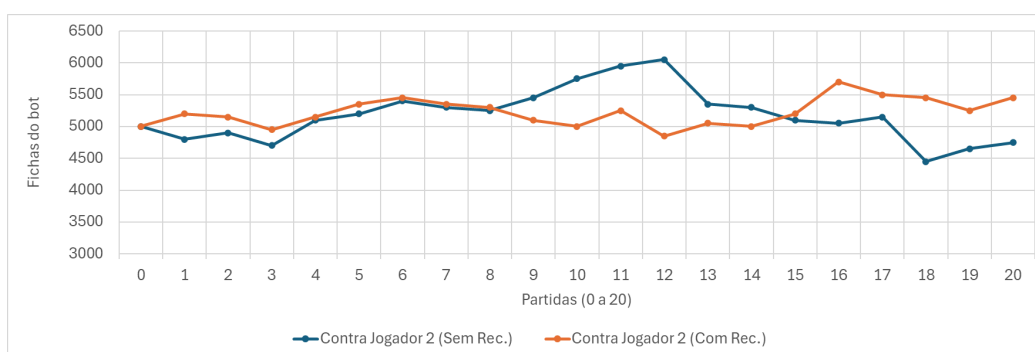


Gráfico 2 – Comparativo de Desempenho contra Jogador 2 (Com Reconhecimento Facial vs. Sem Reconhecimento Facial)

Fonte: Autores

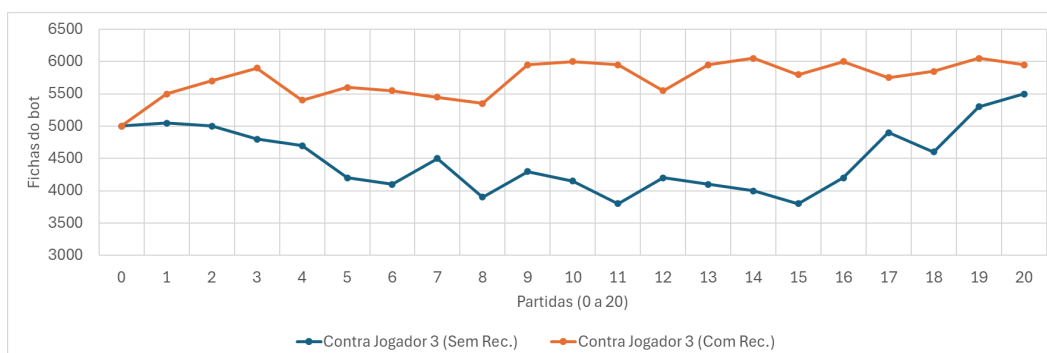


Gráfico 1 – Comparativo de Desempenho contra Jogador 3 (Com Reconhecimento Facial vs. Sem Reconhecimento Facial)

Fonte: Autores

O Gráfico 3 (pessoa A) apresenta a evidência mais clara do potencial do $L(f)$. No cenário B, a linha azul demonstra um desempenho consistentemente negativo, permanecendo sempre abaixo do saldo inicial de 5.000 fichas e terminando com o maior prejuízo da simulação. No cenário A, a linha laranja se mantém sempre acima das 5.000 fichas.

Já o Gráfico 1 demonstra o impacto do módulo na estabilidade dos ganhos a qual o *bot* passou a maior parte da simulação em território negativo, abaixo de 5.000 fichas, conseguindo recuperar o saldo apenas nas partidas finais. No cenário A, o *bot* se manteve em território positivo (acima de 5.000), mostrando uma vantagem mais estável.

E, por fim, o Gráfico 2 apresenta um cenário de impacto neutro. As linhas de ambos os cenários permanecem muito próximas e se cruzam duas vezes, indicando que, contra este autor, o módulo de leitura de expressões não proporcionou uma vantagem estratégica tão decisiva. No entanto, mesmo neste cenário de paridade, sugere que o módulo, no mínimo, não prejudicou o desempenho e ofereceu um leve benefício.

4.3. Análise Qualitativa de Decisões-Chave

A Tabela 5 mostra testes que exemplificam momentos onde a variável $L(f)$ foi o fator decisivo para alterar a jogada do *bot*, justificando os resultados vistos nos gráficos.

Rodada	Mão do Bot	Mão do Jogador	Mesa	Pmc	Emoção detectada	$L(f)$	D simulado (sem $L(f)$)	Decisão Simulada	D final (com $L(f)$)	Decisão final
Flop	4S,AS	QH,JC	KH,5C,JS	53,2	Feliz	-30	63,2	Call	33,2	Fold
Flop	2D,JS	9S,2C	9H,QC,10H	50,16	Feliz	-30	60,16	Call	30,16	Fold
River	9D,7S	5H,JS	8D,9S,5C,9C,KH	95,49	Feliz	-30	105,49	Call	75,49	Call
Turn	KC,2D	KS,9C	JH,4H,AH,10H	34,3	Bravo	45	34,3	Check	79,3	Bet
River	9C,QH	9H,6S	10C,10H,10D,JS,7H	36,9	Medo	35	46,9	Fold	81,9	Call

Tabela 5 – Exemplos de Decisões Influenciadas pela Leitura Facial

Fonte: Autores

As duas primeiras linhas demonstram a eficácia do módulo de leitura em evitar perdas: em ambos os casos, a detecção de felicidade ou surpresa, emoções que indicam cartas boas, reverteu uma decisão que seria *Call* contra a aposta do jogador para um *Fold* correto. Em contraste, a terceira linha mostra a robustez da fórmula, onde uma

probabilidade de Monte Carlo muita alta se sobrepôs a uma emoção negativa para o *bot* (felicidade), mantendo a decisão de *Call* e ganhando o pote. Já a quarta linha demonstra uma ótima leitura de jogo: a detecção de raiva reverteu uma decisão que simulada seria *Check* para um *Bet* agressivo levando à vitória ao final da partida. Por fim quinta a linha demonstra uma captura de blefe: a detecção de medo reverteu uma decisão que simulada seria *Fold* para um *Call*

5. Conclusão

Este trabalho atingiu seus dois objetivos centrais. Primeiro, foi desenvolvida uma Inteligência Artificial funcional para pôquer que foi capaz de integrar probabilidade e leitura de expressões em uma fórmula de decisão modular. O segundo objetivo foi demonstrar que esta integração melhora a tomada de decisão.

Embora uma amostragem de testes maior seja necessária para uma validação estatística completa, a análise comparativa (Tabelas 3 e 4) mostrou que o *bot* operando com reconhecimento facial terminou com saldo positivo contra todos os autores, enquanto em simulação sem reconhecimento facial, terminou com saldo negativo contra a maioria. A análise qualitativa detalhou o porquê dessa diferença, mostrando como o $L(f)$ permitiu ao *bot* evitar perdas (revertendo decisões como de *Calls* para *Folds* ao detectar emoções como de felicidade ou surpresa) e capturar blefes (revertendo *Folds* para *Calls* ao detectar *fear*).

Para trabalhos futuros, se recomenda a expansão dos testes contra um número maior de jogadores, assim como o refinamento da fórmula de decisão, reintroduzindo as variáveis de agressividade e de diferença de aposta em *Blinds*, além de implementar pesos dinâmicos para $L(f)$ e V que possam ser calibrados por Aprendizado de Máquina. Além disso, o módulo de reconhecimento facial pode ser aprimorado para focar em micro expressões baseadas em Paul Ekman (1992) que demonstra que emoções verdadeiras "vazam" por meio de expressões faciais involuntárias e de curta duração, revelando sentimentos que o indivíduo tenta esconder (por exemplo a *poker face*), e para classificar o perfil dos oponentes em tempo real.

Este trabalho, portanto, contribui estabelecendo uma base para o desenvolvimento de inteligências artificiais que sejam não apenas analíticas mas perceptivas também. Conclui-se que o módulo $L(f)$ foi a variável que transformou o resultado de prejuízo em lucro e essa contribuição se mostrou significativa, pois mostra que a IA conseguiu integrar uma camada de percepção psicológica, ao invés de tomar decisões baseadas somente em cálculos matemáticos.

Referências bibliográficas

ALHANAEE, K. et al. Face recognition smart attendance system using deep transfer learning. *Procedia Computer Science*, Elsevier, v. 192, p. 4093–4102, 2021. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1877050921019232>. Acesso em: 28 de março de 2025.

BROWN, N.; SANDHOLM, T. Superhuman ai for multiplayer poker. Science, American Association for the Advancement of Science (AAAS), v. 365, n. 6456, p. 885–890, 2019. Disponível em: <https://www.science.org/doi/abs/10.1126/science.aay2400>. Acesso em: 28 de março de 2025.

CHASLOT, Guillaume M. J.-B. et al. Progressive strategies for Monte-Carlo Tree Search. New Mathematics and Natural Computation, v. 4, n. 3, p. 343–357, 2008. Disponível em: <https://doi.org/10.1142/S1793005708001094>. Acesso em: 04 de dezembro de 2025.

COULOM, R. Efficient selectivity and backup operators in Monte-Carlo Tree Search. In: Proceedings of the 5th international conference on computers and games. Springer, 2006. p. 72–83. Disponível em: https://link.springer.com/chapter/10.1007/978-3-540-75538-8_7. Acesso em: 12 de maio de 2025.

EKMAN, P. An argument for basic emotions. Cognition & Emotion, v. 6, n. 3-4, p. 169-200, 1992. Disponível em: <https://www.tandfonline.com/doi/abs/10.1080/02699939208411068> acesso em: 07 de novembro de 2025

FEINLAND, JACOB ET AL. Poker bluff detection dataset based on facial analysis. In: International conference on image analysis and processing. Cham: Springer International Publishing, 2022. p. 400-410. Disponível em: https://link.springer.com/chapter/10.1007/978-3-031-06433-3_34 acesso em: 14 de maio de 2025

JIN, B.; CRUZ, L.; GONÇALVES, N. Deep facial diagnosis: deep transfer learning from face recognition to facial diagnosis. IEEE Access, Institute of Electrical and Electronics Engineers (IEEE), v. 8, p. 123649–123661, jun. 2020. Disponível em: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9127907>. Acesso em: 28 de março de 2025.

GELLY, S.; WANG, Y.; MUNOS, R.; TEYTAUD, O. Modification of UCT with patterns in Monte-Carlo Go. INRIA, Technical Report 6062, 2006. Disponível em: <https://hal.science/inria-00117266/document>. Acesso em: 04 de dezembro de 2025.

JOFILSAN, N. C. et al. Classificação de gamers por padrões em suas expressões faciais. In: Proceedings of SBGames 2021. Sociedade Brasileira de Computação (SBC), 2021. Disponível em: <https://www.sbgames.org/proceedings2021/IndustriaFull/218855.pdf>. Acesso em: 28 de março de 2025.

KOCSIS, L.; SZEPESVÁRI, C. Bandit based Monte-Carlo planning. In: European Conference on Machine Learning. Springer, 2006. p. 282–293. Disponível em: https://link.springer.com/chapter/10.1007/11871842_29. Acesso em: 12 maio de 2025.

LIMA, T. A. de; MADEIRA, C. A. G. An investigation of using monte-carlo tree search for creating the intelligent elements of rise of mitra. In: Proceedings of SBGames 2017. Sociedade Brasileira de Computação (SBC), 2017. Disponível em: <https://www.sbgames.org/sbgames2017/papers/ComputacaoShort/175198.pdf>. Acesso em: 28 de março de 2025.

PEI, J.; SHAN, P. A micro-expression recognition algorithm for students in classroom learning based on convolutional neural network. *Traitement du Signal*, International Information and Engineering Technology Association (IIETA), v. 36, n. 6, dez. 2019. Disponível em: <https://www.iieta.org/journals/ts/paper/10.18280/ts.360611>. Acesso em: 28 de março de 2025.

RUSSELL, James A. A circumplex model of affect. In: *Journal of personality and social psychology*, v. 39, n. 6, p. 1161, 1980. Disponível em: <https://pdodds.w3.uvm.edu/research/papers/others/1980/russell1980a.pdf>. Acesso em: 23 de outubro 2025

SANTOS, E. H. BERNARDINO, H. S. Redundant action avoidance and non-defeat policy in the monte carlo tree search algorithm for general video game playing. In: Proceedings of SBGames 2017. Sociedade Brasileira de Computação (SBC), 2017. Disponível em: <https://www.sbgames.org/sbgames2017/papers/ComputacaoShort/175320.pdf>. Acesso em: 28 de março de 2025.

SARAIVA JÚNIOR, A. F.; TABOSA, C. M.; COSTA, R. P. da. Simulação de Monte Carlo aplicada à análise econômica de pedido. *Produção*, São Paulo, v. 21, n. 1, p. 149-164, 2011. Disponível em: <https://www.scielo.br/j/prod/a/pSr3JmvpNgDWYkptGpDBfvc/>. Acesso em: 04 de dezembro de 2025.

SERENGIL, S. I.; OZPINAR, A. A Benchmark of Facial Recognition Pipelines and Co-Usability Performances of Modules. *Journal of Information Technologies*, 17(2), 95–107, 2020. Disponível em: <https://doi.org/10.17671/gazibtd.1399077>. Acesso em: 6 de abril de 2025.